and then use the 'save' method. Let's make things a little more interesting then, let's modify multiple articles at the same time.

Let's saw we have a lot of faith in author's whole name starts with a 'K' — why wouldn't you — and we'd like to publish all articles written by such authors. We know how to write this query, we did that in the previous section:

```
Article.objects.filter(author__first_name__startswith='K')
```

A brute force way would be to use use a loop to go though this list of articles, individually set the 'publish_status' to True and then save the article. However there is a better way, and it is as simple as the following:

```
Article.objects.filter(author__first_name__startswith='K').update(publish_status=True)
```

If you execute this query in the shell, you will see a number sow up. This is the number of rows that were affected by this update.

Creating the Models for our CMS

Now that we know how models can be created and used, let's go about creating all the ones we need. At this point the only thing we need to keep in mind is the fact that some of our models need other models in order to work.

For instance, articles need to have tags and categories. In the model for article that we created in the previous section, we left these out. In this section we will update the model we made before to include support for tags and categories. Before that though we need to create models for tags and categories.

So let's start with the simplest, the tag. Here is the model for tags, you should find it entirely unsurprising:

```
class Tag(models.Model):
    name = models.CharField(max_length=32)
    slug = models.SlugField()

    def __str__(self):
        return self.name
```

The categories model is likewise quite simple:

```
class Category(models.Model):
    name = models.CharField(max_length=32)
    slug = models.SlugField()
    description = models.TextField()
```